

NATIONAL UNIVERSITY
of Computer & Emerging Sciences, Lahore

Project Phase 3 (Track C)

Memory Priority Scheduling Algorithm for Cloud Data Center Based on Machine Learning Dynamic Clustering Algorithm

---

Course, namely Artificial Intelligence (AI-2002)

Instructor: Ms. Hajra Waheed

Group ID: 3

Group Members:

Mustafa Salman	24L-3051
Mahrukh Rehman	24L-3037
Bilal Ahmed	24L-3090
Rubia Tahir	24L-3098

1. Algorithm Selection and Justification

1.1. Problem:

A cloud data center must place incoming virtual machines (VMs) on physical machines (PMs) so that as few PMs as possible stay on, which saves energy, to a significant degree; the paper is arguing that typical schedulers optimise CPU utilisation and neglect memory, leaving PMs unbalanced [1], broadly speaking. Memory was already prioritised by the authors' earlier work when mapping tasks and rolling out VMs [2].

1.2. Chosen algorithm:

PMPD (memory priority scheduling with dynamic clustering). It rules [1]:

- **Rule 1, dynamic clustering.** The clusters are, of course, refreshed as VMs are rolled out; the active PMs are grouped with K-means [3] (k = 3) on their (CPU utilisation, memory utilisation), in many respects.
- **Rule 2, VM deployment.** A new VM goes to the cluster whose CPU/memory imbalance it's correcting best; within that cluster it goes to the PM whose remaining running time is normally closest to the VM's duration, so the VM doesn't keep the PM awake longer, in actual application. A PM with no VMs goes to sleep.

1.3. Why this fits:

K-means is an unsupervised method, which suits this issue, because there are no labelled "correct placements" to learn from, on the whole; with two attributes and k = 3 it's cheap enough to re-run during the scheduling, in real-world scenarios. And its clusters are easy to construe. Only a small part of PMPD is machine learning; most of the gain is meant to come from the placement rules built on the clustering; indeed, reinforcement-learning schedulers mentioned in the paper [1] would need a training environment and reward design, which is out of scope for this project.

1.4. Baseline:

First-Fit (place on the first PM that fits) is the paper's simplest comparison [1]. The paper's other baselines (WS [4], TPO [5], DE-ERPSO [6]) were not implemented, so only the First-Fit comparison can be reproduced.

2. Implementation details

Component	Choice
Language	Python 3 (Jupyter / Google Colab)
Numerics and data	numpy [7], pandas [8]
Clustering	scikit-learn [9] `KMeans`
Plots	matplotlib [10]
Standard library	`heapq` (event queue for VM completions), `tarfile` (streaming extraction of the trace)

Modelling assumptions (the paper validates PMPD in CloudSim [11] but does not publish its configuration [1]):

- PM types are following the paper's Table IX [1]: 18 cores/72 GB, 64 cores/256 GB and 16 cores/64 GB, 200 of each type for the synthetic runs, on the whole.
- Synthetic VMs have 1, 2 or 4 vCPUs with memory at 1:2, 1:4 or 1:8 GB per vCPU, durations of 500 to 5000 s and Poisson arrivals at 1 VM per second, on the whole.
- Light-load time is the total active PM time below the minimum-utilisation threshold; in turn, PM utilisation is the mean of CPU, the central processor, and memory utilisation, on the whole.
- Power is linear in CPU use (idle 50 + 1.0 x cores W, peak 100 + 4.0 x cores W); this is our assumption, not the paper's, broadly speaking.
- Both algorithms wake the first sleeping PM that's fitting when no active PM can, to some extent, take the VM, so only the placement logic differs, in real-world scenarios.
- The paper doesn't say how the utilisation threshold enters placement; in turn, before employing the remaining-time rule, in Rule 2 we prefer PMs below the threshold, in real-world scenarios.

3. Code workflow (data input to output)

3.1. Input

Either synthetic PMs and VMs created from the paper's coefficients, or the Alibaba Cluster Trace v2018 [12] batch_instance table, to a significant degree; PMs are uniform, with 96 cores and 100 normalised memory units. Rows with missing values, non-positive duration or demand, or demand larger than one PM are removed, to a significant degree; for the trace, each Stopped instance becomes one VM, like arrival = start_time, duration = end_time - start_time, CPU = cpu_avg / 100 cores, memory = mem_avg.

3.2. Simulation loop

VMs are carried out in arrival order. Finished VMs release their resources, the chosen policy picks a PM, the VM is mainly placed, and a PM that becomes empty sleeps, broadly speaking.

3.3. Policy

Either First-Fit, or PMPD (K-means refit on the active PMs, cluster chosen by post-placement imbalance, PM chosen by remaining time), on the whole.

3.4. Metrics (paper Eq. 1 to 4 [1])

Light-load PM time, average CPU utilisation, average memory utilisation and total energy, so Active PM-hours, peak active PMs and the share of VMs placed are also recorded, on the whole.

3.5. Output

Result tables, line and bar plots, and enhancement of PMPD over First-Fit as percentages or percentage points.

3.6. Real-data pool sizing

The PM pool is set to peak happening together demand x 1.2 and then enlarged until First-Fit places every VM, so that both algorithms are checked on the same accepted workload, broadly speaking.

4. Training process and experimental setup

PMPD **supervisedly trains, namely** K-means is fitted online on the current state of the active PMs, in real-world scenarios. There's therefore no train/test split and no epochs. Check is done by modelling instead.

Setting	Value
Clusters (k)	3
Let's start up fit of K-means	k-means++ initialisation [13], `n_init = 3`, `max_iter = 20`, fixed `random_state`
Later refits	warm-started from the previous centres, `n_init = 1`, `max_iter = 10`
Refit frequency	every 25 VM deployments, with every VM re-labelling PMs against the latest centres
Synthetic sweeps	VM count 10,000 to 50,000; minimum-utilisation threshold 0.50, 0.60, 0.70 (paper Table X [1])
Randomness	3 seeds (0, 1, 2); both algorithms are generally getting the same VM stream
Real-data windows	3 windows of 50,000 VMs (offsets 0, 100,000, 200,000) from the first 1,000,000 trace rows
Real-data thresholds	0.2, 0.3, 0.6

A small hand-checkable workload is run first to corroborate that both policies place every VM, never over-commit a PM and end with all PMs asleep, to a significant degree.

5. Initial findings

5.1. Real data (Alibaba v2018, threshold 0.3)

Every reported run placed 100% of VMs. Of the first 1,000,000 trace rows, 907,962 VMs remained after cleaning; their median demand is 0.70 cores and 0.05 memory units (of 100), to a significant degree. As, well, their median duration is 17 s. Negative changes mean PMPD is worse than First-Fit, on the whole.

Window	Pool (PMs)	Algorithm	Energy (kWh)	Active PM-hours	Avg CPU util	Avg mem util	Light-load time (s)
0	41	First-Fit	5.296	18.421	0.419	0.107	35,363
0	41	PMPD	5.349	18.786	0.410	0.105	36,334
1	28	First-Fit	2.678	10.587	0.316	0.150	24,574
1	28	PMPD	2.693	10.691	0.313	0.148	25,358
2	25	First-Fit	1.858	8.010	0.254	0.180	19,296
2	25	PMPD	1.890	8.232	0.247	0.175	20,207

Both algorithms employed the same number of PMs at peak (34, 23 and 21), broadly speaking; indeed, PMPD's energy is 1.0%, 0.6% and 1.7% higher than First-Fit in windows 0, 1 and 2, and its light-load time is 2.8% to 4.7% higher, broadly speaking.

The 34-PM minimum equals the CPU bound (3,237 peak cores / 96 cores per PM = 33.7), so neither algorithm can do better on peak PM count, on the whole. Pool-size sweep (window 0): Shrinking the pool below 34 PMs is making both algorithms reject VMs at the same point, to a significant degree.

5.2. Synthetic data (paper parameters, 3 seeds, 200 PMs of each type)

No VM was left unplaced in any run. The first table deviates the number of VMs at threshold 0.6; the second differs the threshold at 50,000 VMs, in real-world scenarios; and thus, a positive reduction means PMPD is better than First-Fit; for utilisation, a positive change means PMPD is higher, on the whole.

VMs	Light-load time reduction (%)	CPU util change (pp)	Memory util change (pp)	Energy reduction (%)
10,000	32.28	2.05	3.21	2.21
20,000	41.17	0.22	1.13	1.38
30,000	43.75	-0.99	-0.13	1.40
40,000	44.63	-3.10	-0.25	1.65
50,000	49.07	-2.10	-0.76	1.08

Threshold	Light-load time reduction (%)	CPU util change (pp)	Memory util change (pp)	Energy reduction (%)
0.5	61.10	-1.58	-0.42	0.89
0.6	49.07	-2.10	-0.76	1.08
0.7	28.39	-1.91	1.59	3.00

5.3. Explanation

On synthetic data PMPD clearly shortens light-load PM time (28% to 61% less than First-Fit) and cuts energy by 1% to 3%, but it does not raise average utilisation as the paper reports [1]: CPU utilisation is about 1 to 3 points lower from 30,000 VMs upwards, and memory utilisation differs by under 1 point there, except +1.6 points at threshold 0.7.

The direction of the light-load result is agreeing with the paper, which reports reductions of 67% against First-Fit as the VM count is growing and 72%, across the thresholds [1], in real-world scenarios; in fact, though, our scales are smaller. Our energy savings are far below its roughly 20% [1]. How about this, the energy differences also look small relative to the seed-to-seed spread in the sweep plots, so they're indicative only, in many respects.

A likely explanation, not tested, is that the trace's VMs are tiny and short (median 17 s) relative to 96-core PMs and memory is slack (11% to 18% utilisation, against 25% to 42% for CPU), so there's little CPU/memory imbalance to correct and First-Fit already packs close to the CPU lower bound, in many respects. On the Alibaba trace no benefit is given by PMPD and costs about 1% to 2% more energy, in many respects.

As, the paper's CloudSim set-up, power model and durations are unpublished, findings should be verified with it in direction rather than in exact percentages, in actual application.

6. Obstacles faced

6.1. Under-specified paper:

The CloudSim configuration, power model, trace tables and the function of the utilisation threshold aren't published, so we made definite assumptions and recorded them, in many respects.

6.2. Trace units

CPU and memory units in `batch_instance` were construed from the trace's conventions and should be inspected against its schema document, in actual application.

6.3. Pool sizing

The calibration procedure above fixed this. The first real-data run sized the pool from mean demand and left about 30% of VMs unplaced (30,604 for First-Fit and 30,583 for PMPD) with every PM always on, to a significant degree.

6.4. Window edge effects

Windows starting mid-trace begin with an empty data center, because VMs that started earlier are missing, which depresses utilisation early in each window for both algorithms, broadly speaking.

6.5. Runtime

PMPD took about 22 s against 2 s for First-Fit on 100,000 VMs, because of the repeated K-means refits, on the whole. 193 s and 188 s. Were, for all practical purposes, taken by the full 3-seed synthetic sweeps

6.6. Large and partial trace files

The trace is pulled by streaming only the first rows, so even so, on top of that, the reader tolerates a partially downloaded archive, in many respects.

References

[1] B. Liang and D. Wu, "Memory priority scheduling algorithm for cloud data center based on machine learning dynamic clustering algorithm," *IEEE Trans Ind Informat.* vol. 21, no. 4, pp. 3485-3492, Apr. 2025, doi: 10.1109/TII.2025.3528574.

[2] B. Liang, X. Dong, Y. Wang, and X. Zhang, "Memory-aware resource management algorithm for low-energy cloud data centers," *Future Gener. Comput. Syst.* vol. 113, pp. 329-342, 2020.

[3] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129-137, Ruin. 1982.

[4] K. M. Tarplee, A. A. Maciejewski, and H. J. Siegel, "Robust performance-based resource provisioning using a steady-state model for multi-objective stochastic programming," *IEEE Trans. Cloud Comput.* vol. 7, no. 4, pp. 1068-1081, Oct. Dec. 2019.

[5] F. López-Pires, B. Barán, L. Benítez, S. Zalimben, and A. Amarilla, "Virtual machine placement for elastic infrastructures in overbooked cloud computing datacenters under uncertainty," *Future Gener Comput Syst.* vol. 79, pp. 830-848, 2018.

[6] J. Lin, W. Lin, W. Wu, W. Lin, and K. Li, "Energy-aware virtual machine placement based on a holistic thermal model for cloud data centers," *Future Gener. Comput. Syst.* vol. 161, pp. 302-314, 2024.

[7] C. R. Harris et al. "Array programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.

[8] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python Sci. Conf.* 2010, pp. 56-61.

[9] F. Pedregosa et al. "Scikit-learn: Machine learning in Python," *J. Mach Learn Res.* vol. 12, pp. 2825-2830, 2011.

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput Sci Eng.* vol. 9, no. 3, pp. 90-95, 2007.

[11] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms," *Softw Pract Exper.* vol. 41, no. 1, pp. 23-50, 2011.

[12] Alibaba Cluster Trace Program, "Alibaba cluster trace v2018," GitHub repository. [Online]. Available: https://github.com/alibaba/clusterdata

[13] D. Arthur and S. Vassilvitskii, "k-means++: The advantages of careful seeding," in *Proc. 18th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2007, pp. 1027-1035.